```
# Name : Aishwarya Laxmikant Lonikar
# CS AI - D
# Roll No. 2
# Subject : Machine Learning
# PRN : 12410541
```

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
```

```
np.random.seed(42)

brands = ["Lakme", "Maybelline", "L'Oreal", "MAC", "Nykaa"]
categories = ["Lips", "Face", "Eyes"]

product_names = [
    "Lipstick", "Foundation", "Mascara",
    "Compact", "Eyeliner", "Blush"
]

data = pd.DataFrame({
    "Brand": np.random.choice(brands, 200),
    "Product_Name": np.random.choice(product_names, 200),
    "Category": np.random.choice(categories, 200),
    "Age": np.random.randint(18, 60, 200),
    "Income": np.random.randint(20000, 100000, 200),
    "Previous_Purchases": np.random.randint(0, 20, 200)
})

# Create target variable logically
data["Purchased"] = (
    (data["Previous_Purchases"] > 8) |
    (data["Income"] > 60000)
).astype(int)
```

```
data.head()
```

	Brand	Product_Name	Category	Age	Income	Previous_Purchases	Purchased
0	MAC	Foundation	Eyes	55	26090	3	0
1	Nykaa	Blush	Face	24	31637	5	0
2	L'Oreal	Blush	Lips	30	42415	0	0
3	Nykaa	Mascara	Face	57	44071	4	0
4	Nykaa	Lipstick	Eyes	59	78243	0	1

Next steps: New interactive sheet

```
data_encoded = pd.get_dummies(data, columns=["Brand", "Product_Name", "Category"], drop_first=True)

data_encoded.head()
```

	Age	Income	Previous_Purchases	Purchased	Brand_Lakme	Brand_MAC	Brand_Maybelline	Brand_Nykaa	Product_Name_Compact	Product_Name_Eyeliner
0	55	26090	3	0	False	True	False	False	False	False
1	24	31637	5	0	False	False	False	True	False	False
2	30	42415	0	0	False	False	False	False	False	False
3	57	44071	4	0	False	False	False	True	False	False
4	59	78243	0	1	False	False	False	True	False	False

Next steps: New interactive sheet

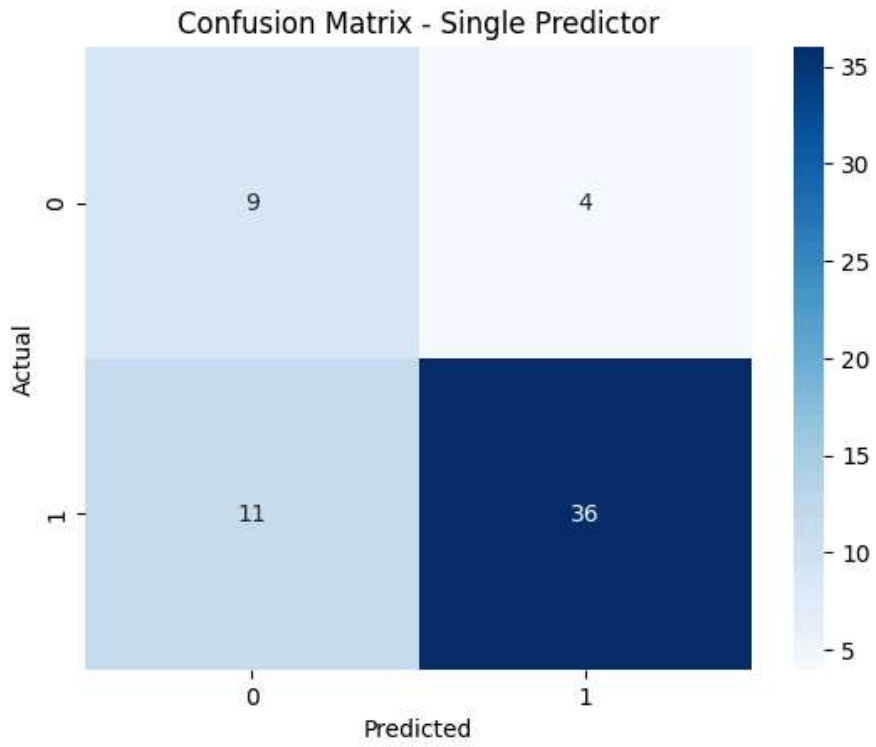
```
X_single = data_encoded[["Previous_Purchases"]]
y = data_encoded["Purchased"]

X_train, X_test, y_train, y_test = train_test_split(
    X_single, y, test_size=0.3, random_state=42
)
```

```
model_single = LogisticRegression()
model_single.fit(X_train, y_train)

y_pred_single = model_single.predict(X_test)
```

```
cm_single = confusion_matrix(y_test, y_pred_single)

print("Confusion Matrix:\n", cm_single)

print("\nAccuracy:", accuracy_score(y_test, y_pred_single))
print("Precision:", precision_score(y_test, y_pred_single))
print("Recall:", recall_score(y_test, y_pred_single))
print("F1 Score:", f1_score(y_test, y_pred_single))
```

```
Confusion Matrix:
 [[ 9  4]
 [11 36]]

Accuracy: 0.75
Precision: 0.9
Recall: 0.7659574468085106
F1 Score: 0.8275862068965517
```

```
sns.heatmap(cm_single, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Single Predictor")
plt.show()
```



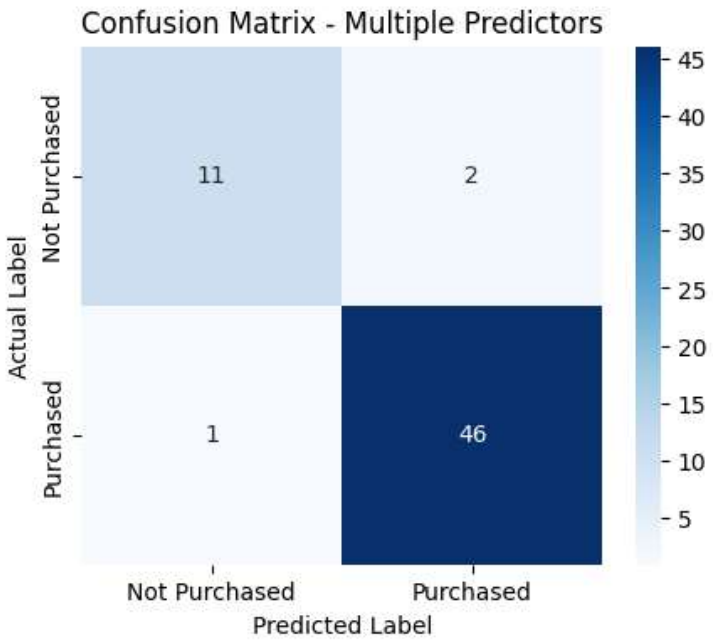

```
X = data_encoded[[
    "Previous_Purchases",
    "Age",
    "Income"
]]

y = data_encoded["Purchased"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model_multi = LogisticRegression(max_iter=1000)
model_multi.fit(X_train, y_train)

y_pred_multi = model_multi.predict(X_test)
```

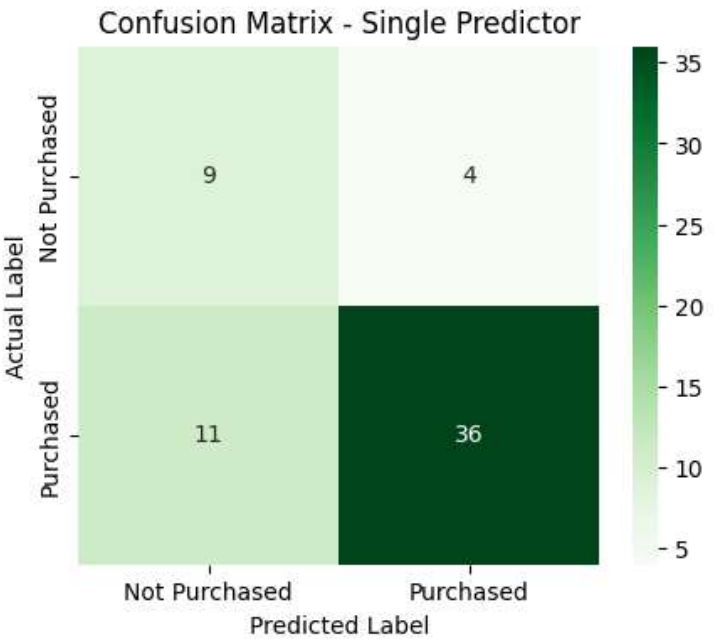
```
cm_multi = confusion_matrix(y_test, y_pred_multi)

print("Confusion Matrix:\n", cm_multi)
print("Accuracy:", accuracy_score(y_test, y_pred_multi))
print("Precision:", precision_score(y_test, y_pred_multi))
print("Recall:", recall_score(y_test, y_pred_multi))
print("F1 Score:", f1_score(y_test, y_pred_multi))
```

```
Confusion Matrix:
 [[11  2]
 [ 1 46]]
Accuracy: 0.95
Precision: 0.9583333333333334
Recall: 0.9787234042553191
F1 Score: 0.968421052631579
```

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(5,4))
sns.heatmap(cm_multi,
            annot=True,
            fmt='d',
            cmap='Blues',
            xticklabels=["Not Purchased", "Purchased"],
            yticklabels=["Not Purchased", "Purchased"])

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Multiple Predictors")
plt.show()
```




```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(5,4))
sns.heatmap(cm_single,
            annot=True,
            fmt='d',
            cmap='Greens',
            xticklabels=["Not Purchased", "Purchased"],
            yticklabels=["Not Purchased", "Purchased"])

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Single Predictor")
plt.show()
```




```
def get_confusion_values(y_true, y_pred):
    TP = TN = FP = FN = 0

    for i in range(len(y_true)):
        if y_true.iloc[i] == 1 and y_pred[i] == 1:
            TP += 1
        elif y_true.iloc[i] == 0 and y_pred[i] == 0:
            TN += 1
        elif y_true.iloc[i] == 0 and y_pred[i] == 1:
            FP += 1
        elif y_true.iloc[i] == 1 and y_pred[i] == 0:
            FN += 1

    return TP, TN, FP, FN
```

```
def calculate_accuracy(y_true, y_pred):
    TP, TN, FP, FN = get_confusion_values(y_true, y_pred)
    return (TP + TN) / (TP + TN + FP + FN)
```

```
def calculate_precision(y_true, y_pred):
    TP, TN, FP, FN = get_confusion_values(y_true, y_pred)
    if (TP + FP) == 0:
        return 0
    return TP / (TP + FP)
```

```
def calculate_recall(y_true, y_pred):
    TP, TN, FP, FN = get_confusion_values(y_true, y_pred)
    if (TP + FN) == 0:
        return 0
    return TP / (TP + FN)
```

```
def calculate_f1(y_true, y_pred):
    precision = calculate_precision(y_true, y_pred)
    recall = calculate_recall(y_true, y_pred)

    if (precision + recall) == 0:
        return 0
    return 2 * (precision * recall) / (precision + recall)
```

```
print("Accuracy:", calculate_accuracy(y_test, y_pred_multi))
print("Precision:", calculate_precision(y_test, y_pred_multi))
print("Recall:", calculate_recall(y_test, y_pred_multi))
print("F1 Score:", calculate_f1(y_test, y_pred_multi))
```

```
Accuracy: 0.95
Precision: 0.9583333333333334
Recall: 0.9787234042553191
F1 Score: 0.968421052631579
```

```
# Get confusion values for SINGLE predictor model
TP_s, TN_s, FP_s, FN_s = get_confusion_values(y_test, y_pred_single)

print("\nConfusion Matrix Values (Single Predictor)")
print("-------------------------------------------")
print(f"TP: {TP_s}")
print(f"TN: {TN_s}")
print(f"FP: {FP_s}")
print(f"FN: {FN_s}")

print("\nModel Performance (Single Predictor)")
print("-------------------------------------------")
print(f"Accuracy : {calculate_accuracy(y_test, y_pred_single):.4f}")
```